

Archetypes

Raphaël Péné

Table des matières

1 Les archetypes	1
1.1 Archétypes théoriques	1
1.2 Les Archétypes : les couleurs	6

1 Les archetypes

L'article de Eugster et Leisch (2009) présente le package R **archetypes**. Un archétype, est d'après le Larousse un “*Modèle original ou idéal sur lequel est fait un ouvrage, une œuvre*”. Le travail ici a pour objectif de trouver des points, appelés archétypes, dans un ensemble d'observations multivariées, qui permettent à toutes les observations d'être correctement représentées par une combinaison convexe des archétypes.

1.1 Archétypes théoriques

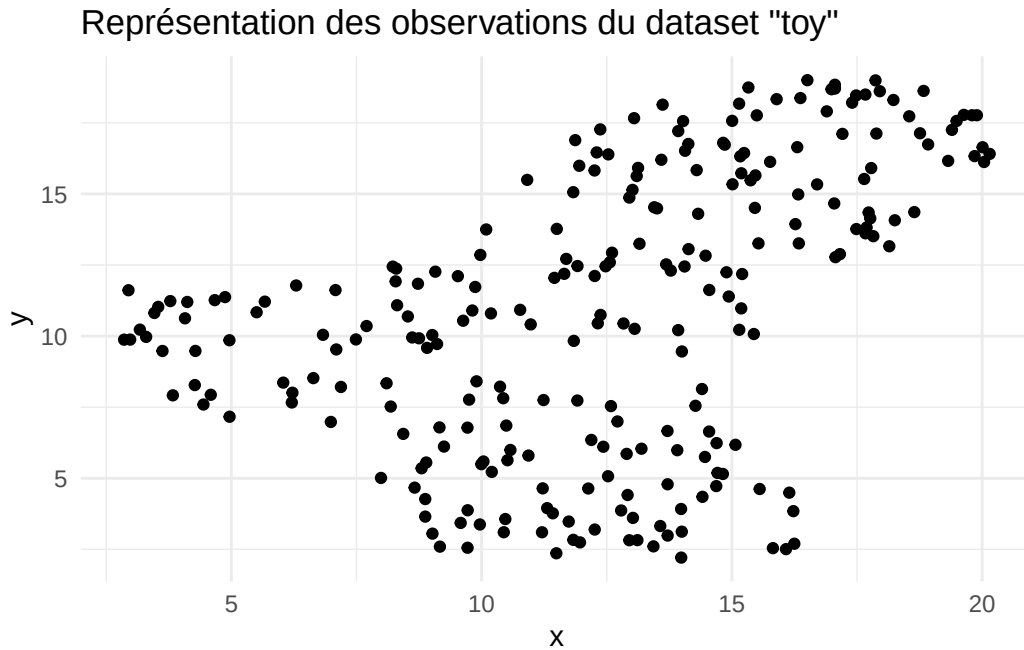
Chargement des librairies R utilisées dans le projet :

```
library(archetypes)
library(ggplot2)
library(imager)
library(tidyverse)
library(plotly)
library(sf)
```

L'article utilise un jeu de données appelé **toy** pour présenter un exemple théorique de la détermination des archétypes. Les observations de **toy** sont définies par 2 coordonnées : x et y.

```
data("toy")
```

```
ggplot(toy,aes(x,y))+  
  geom_point()+  
  ggtitle("Représentation des observations du dataset \"toy\")+  
  theme_minimal()
```



Les archétypes doivent être placés dans l'enveloppe convexe de la distribution des données. En effet, on souhaite que les archétypes soient une combinaison pondérée des individus. La somme des pondérations doivent être égales à 1. On va essayer de trouver le nombre d'archétypes optimal pour minimiser la distance entre les individus et les archétypes.

Résultat de théorème : à partir de 2 archétypes, les archétypes doivent être sur la bordure de l'enveloppe convexe. Dans l'algorithme, les archétypes sont placés aléatoirement dans l'enveloppe et les itérations suivantes déplacent les archétypes vers les extrémités afin de minimiser les distances. L'algorithme ne connaît pas a priori la forme de la distribution des individus. Attention, ici, comme avec les k-means, les archétypes trouvés peuvent être un sous-optimum.

On relie les archétypes entre eux par des segments. Les observations sont projetés sur les segments tracés entre les archétypes. Les points à l'intérieur de l'enveloppe convexe sont une combinaison des 3 archétypes. Ceux à l'extérieur doivent être projeté. A la fin de la simulation, le "bruit" sont les individus qui ne sont pas dans l'enveloppe. Pour avoir un bruit égale à 0, il faudrait avoir le même nombre d'archétypes que le nombre de points sur l'enveloppe convexe.

```
# on utilise la même seed que dans l'article
set.seed(1986)

# On utilise la fonction archetypes sur le dataset toy, on spécifie le nombre
#d'archetypes voulu à 3
a <- archetypes(toy, 3, verbose = TRUE)
```

```
1: rss = 0.04406432, improvement = 0.07474196
2: rss = 0.04169600, improvement = 0.00236832
3: rss = 0.03907572, improvement = 0.00262028
4: rss = 0.03709453, improvement = 0.00198119
5: rss = 0.03512387, improvement = 0.00197065
6: rss = 0.03393902, improvement = 0.00118485
7: rss = 0.03175524, improvement = 0.00218379
8: rss = 0.02731173, improvement = 0.00444351
9: rss = 0.02055071, improvement = 0.00676101
10: rss = 0.01403723, improvement = 0.00651348
11: rss = 0.01130886, improvement = 0.00272837
12: rss = 0.00977788, improvement = 0.00153098
13: rss = 0.00869493, improvement = 0.00108294
14: rss = 0.00808497, improvement = 0.00060997
15: rss = 0.00774999, improvement = 0.00033498
16: rss = 0.00756613, improvement = 0.00018386
17: rss = 0.00745491, improvement = 0.00011123
18: rss = 0.00738143, improvement = 0.00007348
19: rss = 0.00733080, improvement = 0.00005063
20: rss = 0.00729353, improvement = 0.00003727
21: rss = 0.00728053, improvement = 0.00001300
22: rss = 0.00727438, improvement = 0.00000615
23: rss = 0.00727114, improvement = 0.00000324
24: rss = 0.00726871, improvement = 0.00000243
25: rss = 0.00726710, improvement = 0.00000160
26: rss = 0.00726644, improvement = 0.00000066
27: rss = 0.00726864, improvement = -0.00000220
```

```
a
```

Archetypes object

```
archetypes(data = toy, k = 3, verbose = TRUE)
```

Convergence after 27 iterations
with RSS = 0.007268644.

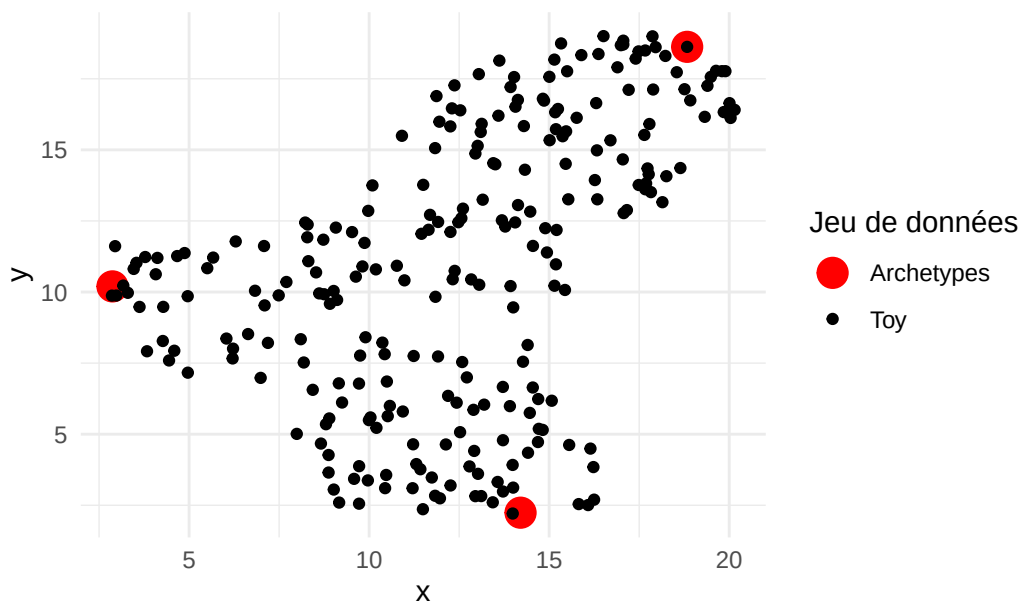
Chaque itération de la fonction a pour objectif de minimiser le RSS. Lorsque l'itération suivante fait passer le RSS en négatif, l'algorithme se stoppe et les coordonnées des archétypes sont trouvées.

```
#les coordonnées des archétypes  
a$archetypes
```

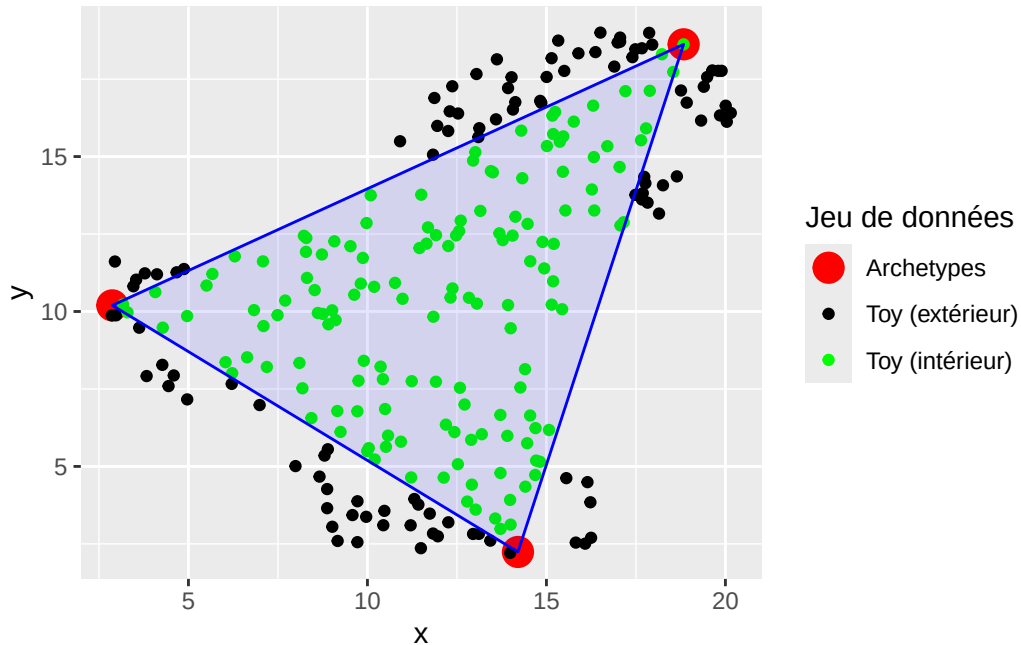
```
      x      y  
[1,] 18.832121 18.623102  
[2,] 14.204922  2.239856  
[3,]  2.872474 10.197810
```

On peut maintenant les représenter dans le plan avec les autres observations :

Représentation des archetypes du dataset "toy"



Maintenant on peut visualiser les observations qui sont dans l'enveloppe convexe :



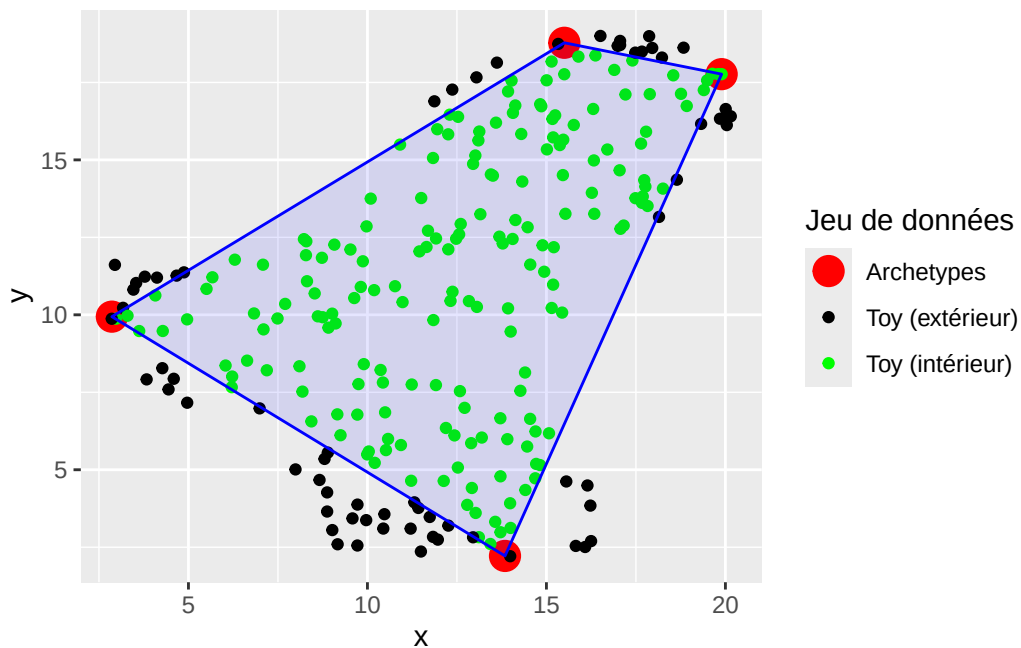
On peut essayer avec un nombre d'archetypes arbitrairement plus grand. Avec 4 archetypes :

```
set.seed(1986)
b <- archetypes(toy, 4, verbose = TRUE)
```

1: rss = 0.03661230, improvement = 0.05241537
2: rss = 0.03356227, improvement = 0.00305003
3: rss = 0.03009807, improvement = 0.00346420
4: rss = 0.02367849, improvement = 0.00641958
5: rss = 0.01525037, improvement = 0.00842812
6: rss = 0.01065077, improvement = 0.00459960
7: rss = 0.00871585, improvement = 0.00193492
8: rss = 0.00750293, improvement = 0.00121293
9: rss = 0.00665149, improvement = 0.00085144
10: rss = 0.00607836, improvement = 0.00057313
11: rss = 0.00571690, improvement = 0.00036146
12: rss = 0.00550005, improvement = 0.00021685
13: rss = 0.00536954, improvement = 0.00013051
14: rss = 0.00529071, improvement = 0.00007884
15: rss = 0.00525116, improvement = 0.00003955
16: rss = 0.00523391, improvement = 0.00001725
17: rss = 0.00523046, improvement = 0.00000344
18: rss = 0.00523763, improvement = -0.00000717

```
#voici le nombre d'itération pour obtenir le plus petit RSS
b$archetypes
```

```
      x      y
[1,] 19.894904 17.767205
[2,]  2.860161  9.941252
[3,] 15.500106 18.779746
[4,] 13.842473  2.218635
```



1.2 Les Archétypes : les couleurs

Maintenant que nous avons réalisé le projet théorique, nous prenons un cas pratique avec un tableau. Nous souhaitons trouver les archétypes de couleur du tableau de Van Gogh **La nuit étoilée**. Nous transformons l'image du tableau en data frame. Chaque ligne représente un pixel de l'image et la valeur de couleur associée. Les couleurs sont décrites par le triplet RGB. Soit rouge R (1), vert G (2), et bleu B (3). Par pixel nous avons donc 3 lignes pour décrire sa couleur. En faisant un pivot wider on retrouve une ligne par pixel avec les valeurs des couleurs RGB dans 3 colonnes distinctes.

```
img<-load.image('nuit_etoilée.jpeg')
```

```
df <- as.data.frame(img)
```

```
df_rgb <- df |> pivot_wider(names_from=cc, values_from = value)
```

```
df_rgb <- df_rgb |> rename(R='1',G='2',B='3')
```

```
names(df_rgb)
```

```
[1] "x" "y" "R" "G" "B"
```

```
head(df_rgb,10)
```

```
# A tibble: 10 x 5
```

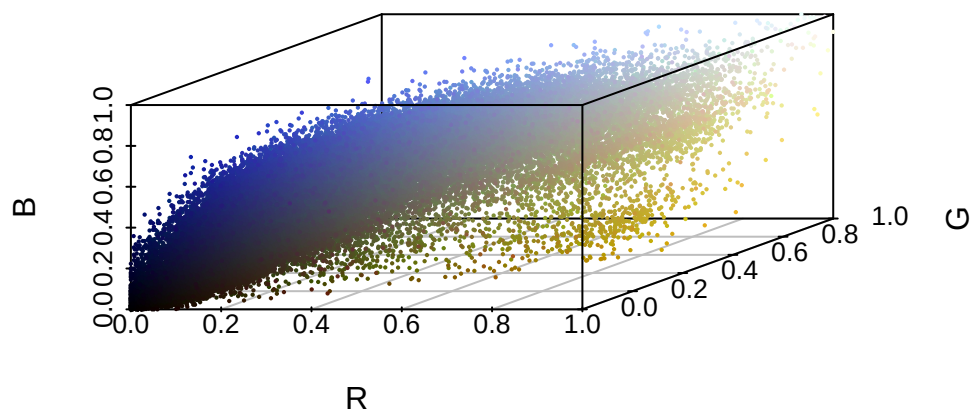
	x	y	R	G	B
	<int>	<int>	<dbl>	<dbl>	<dbl>
1	1	1	0.357	0.333	0.192
2	2	1	0.255	0.220	0.192
3	3	1	0.278	0.247	0.306
4	4	1	0.141	0.125	0.184
5	5	1	0.173	0.176	0.192
6	6	1	0.106	0.125	0.149
7	7	1	0.102	0.114	0.188
8	8	1	0.0863	0.0941	0.192
9	9	1	0.141	0.149	0.247
10	10	1	0.157	0.157	0.267

Représentation en 3D d'un échantillon de 50 000 pixels issus du tableau projetés sur les colonnes R, G et B.

file:///C:/Users/tenep/AppData/Local/Temp/Rtmpg1NHZB/file6e003e2c5d0e/widget6e003a75390a.htm

Distribution des couleurs de 50 000 pixels du tableau

Distribution des couleurs de 50 000 pixels du tableau



Eugster, Manuel J. A., et Friedrich Leisch. 2009. « From Spider-Man to Hero — Archetypal Analysis in R ». *Journal of Statistical Software* 30 (8) : 1-23. <https://doi.org/10.18637/jss.v030.i08>.